
Conventions d'écriture de code Java

Ce document rassemble quelques conventions usuelles d'écriture de code Java. Il est inspiré des recommandations émises par Sun [1].

Il est important de noter que ces règles sont bien des *conventions* et ne font pas partie de la syntaxe de Java.

Les objectifs de ces conventions d'écriture sont les suivants :

1. Ecrire des programmes plus lisibles pour les personnes qui doivent relire ou utiliser votre code
2. Accélérer votre propre écriture de code, d'une part en systématisant les choix de nommage et d'autre part en rendant votre code plus durable vis-à-vis de votre propre relecture
3. S'habituer aux conventions qui font foi dans les entreprises produisant du code

Dans le cadre de la réalisation des TP pour le module 632-1, les conventions d'écritures **surlignées en jaune** dans le document sont obligatoires.

1. Langue

Privilégiez l'utilisation de l'Anglais pour l'écriture de votre code. D'une part, les mots clés réservés du langage Java sont en Anglais. D'autre part, l'Anglais est généralement imposé dans toute entreprise impliquée de près ou de loin dans la production de code.

2. Fichiers

Les noms de fichiers de source Java ont le suffixe `.java`. Les noms de fichiers compilés (bytecode) ont le suffixe `.class`.

Chaque fichier source Java contient une seule classe ou interface publique. Le nom du fichier (sans le suffixe `.java`) est *obligatoirement* le nom de cette classe ou interface

publique. Si on y met des classes ou interfaces privées, la classe ou interface publique doit être la première à être définie dans le fichier.

On organisera le contenu de chaque fichier dans l'ordre suivant :

- Bloc de commentaires de début, comportant
 - le contexte d'écriture de la classe, par exemple réponse à la question xyz du TP de la semaine 3
 - le nom de l'auteur ou des auteurs
 - la date de création et éventuellement la dernière mise à jour.
- Déclaration du *package* éventuel de la classe, suivie des importations éventuelles, par exemple :
 - `package dupont.exercices;`
 - `import java.awt.*;`
- Définition de la classe ou de l'interface (s'il y en a plusieurs, la classe ou interface publique en premier). Cette définition se fait dans l'ordre suivant :

Documentation de la classe ou de l'interface (commentaires <code>/** ... */</code>)	cf. § 4
Déclaration de la classe ou de l'interface	Exemple : <code>public class Bank</code>
Commentaires de programmation (<code>/* ... */</code>), si nécessaire, c'est-à-dire commentaires généraux sur la mise en oeuvre de la classe, qui n'ont pas leur place dans la documentation	cf. § 4
Déclaration des variables de classe (<code>static</code>)	D'abord les <code>public</code> , puis les <code>protected</code> , puis les <code>private</code>
Déclaration des variables d'instance	D'abord les <code>public</code> , puis les <code>protected</code> , puis les <code>private</code>
Constructeurs	
Méthodes	Les regrouper de préférence par fonctionnalités

Exemple :

```
/*
 * Exercise 632-1, week 3, question 2
 * Author: C. Haddock
 * Date creation: 18.02.2009
 * Date last modification: 19.02.2009
 */

package hevs.progmod;

import java.util.ArrayList;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * The <code>Mystery</code> class represents ...
 *
 * @author Captain Haddock
 * @see java.lang.StringBuilder
 */
public final class Mystery
{
    private final char value[];
    ...
}
```

3. Indentation

Définissez une unité d'indentation (typiquement 4 espaces) et utilisez là dans tout votre code de façon cohérente. Le niveau d'indentation correspond au degré d'imbrication de votre code.

Soyez également cohérent dans vos alignements d'accolades ouvrantes et fermantes.

Le document SUN recommande de mettre l'accolade ouvrante à la fin de la ligne où commence le bloc concerné, et l'accolade fermante sur une ligne à part, indentée au niveau de l'instruction qui a entraîné l'ouverture du bloc. Les méthodes à corps vide font exception avec l'accolade fermante qui suit immédiatement l'accolade ouvrante.

Exemples :

```
class Test extends Object {
    int var1;
    int var2;

    Test(int i, int j) {
        var1 = i;
        var2 = j;
    }

    int emptyMethod() {}
}
```

```

void compute() {
    ...
    if (var1 > 0) {
        var2--;
        ...
    }
    ...
    while (var1 != var2) {
        ...
    }
}
...
}

```

Évitez les lignes trop longues (typiquement plus de 80 caractères) :

- Vous pouvez toujours écrire une expression très longue sur plusieurs lignes. Dans ce cas, on cherchera à les couper aux endroits qui laissent la plus grande lisibilité (par ex après une virgule, avant un opérateur, au niveau de parenthèse le plus englobant possible). **En cas d'instructions sur plusieurs lignes, veillez à aligner la nouvelle ligne avec le début de l'expression qui est au même niveau à la ligne précédente.**
- Vous pouvez également augmenter la lisibilité d'une instruction compliquée en utilisant des variables intermédiaires.

De façon générale, faites preuve de bon sens. Si vous avez vous-même des difficultés à lire une instruction, c'est un signe qu'il faut la reformater ou utiliser des variables intermédiaires pour rendre le code plus lisible.

Exemples :

```

fonc(longExpression1, longExpression2, longExpression3,
     longExpression4, longExpression5);

```

```

var = fonc1(veryLongExpression1,
            fonc2(veryLongExpression2,
                  veryLongExpression3));

```

```

taxToPay = captainAge * (length + width - height)
            + 4 * shirtSize;

```

Pour les instructions `if`, s'il faut présenter la condition sur plusieurs lignes, indenter de 8 espaces au lieu de 4 pour que l'instruction qui suit reste lisible :

```

if ((condition1 && condition2)
    || (condition3 && condition4)
    || (condition5 && condition6)) {
    doSomething()    // remains ok to read
    ...
}

```

4. Commentaires

4.1. Commentaires de programmation

On les utilise soit pour "masquer" une partie du code, en cours de développement, soit pour expliquer certains détails. Ils peuvent être mis en œuvre différemment :

1. **Blocs de commentaires** : servent à expliciter des fichiers, des méthodes, des structures de données et des algorithmes. Les précéder d'une ligne blanche, et les présenter comme suit :

```
/*  
 * This is a block of comments.  
 * It can be on several lines, each beginning with a star character.  
 * Stars are aligned in a column by using a space before and after the star.  
*/
```

2. **Une ligne de commentaire** : si votre commentaire ne tient pas sur une ligne, ne faites pas suivre deux lignes de commentaires ; utilisez de préférence un bloc de commentaires. Exemple:

```
if (condition) {  
    /* Single line of comment */  
    ....  
}
```

3. **Commentaires de fin de ligne** : ils peuvent prendre les deux formes `/* ... */` ou `// ...`, la deuxième servant en outre à "masquer" une partie du code en cours de développement. Exemples :

```
if (a == 2) {  
    return TRUE;      /* in-line block comment */  
}  
else if (toto > 1) {  
    // in-line comment using double slash  
    ...  
}  
else  
    return FALSE;     // in-line comment  
  
// if (t > 1) {  
//  
//     // this code is commented out as it need to be validated  
//     ...  
//}  
//else  
//    return FALSE;
```

4.2. Commentaires de documentation

Ces commentaires sont, comme les autres, ignorés par le compilateur. Ils sont par contre interprétés par l'outil `javadoc`, qui permet de créer des pages html de

documentation de vos programmes. Pour une introduction à javadoc, voir son site de référence [2], où vous trouverez aussi un manuel avec des exemples. Un commentaire de documentation commence par `/**` et se termine par `*/`.

De façon générale, il est recommandé de **commenter la classe** (cf exemple « Specialisation of the parent class Client » ci-dessous) et tout ce qui est public (méthode, variable d'instance, variable de classe) car ce sont ces éléments qui sont visibles depuis l'« extérieur » d'une classe. Cette recommandation est généralement imposée dans les compagnies actives en informatique.

Exemples de commentaires de documentation :

Pour une classe :

```
/**
 * Specialisation of the parent class Client.
 * @author    John Smith
 * @see       Client
 */
class RetailClient extends Client
{
    ...
}
```

Pour une méthode :

```
/**
 * Return the amount after application of the
 * value-added tax.
 * @param amountBeforeTax the amount before taxation
 * @return amount after tax
 */

public double addVAT(double amount) {
    ...
}
```

5. Déclarations et instructions

Il est recommandé de **ne mettre qu'une déclaration de variable par ligne**, et de regrouper les déclarations en début de bloc. Essayez d'initialiser les variables locales dès leur déclaration (il y a bien sûr des cas où cela est difficile ou impossible, comme quand l'initialisation dépend d'un calcul préalable).

Par souci de clarté, évitez de masquer des variables à un certain niveau par des variables de même nom à un niveau d'imbrication inférieur, comme dans :

```
int total;
...
fonction() {
```

```

    if (condition) {
        int total;        // avoid this !!
        ...
    }
    ...
}

```

Mettez une seule instruction par ligne ; évitez par exemple :

```

compte++; valeur -= compte;    // avoid this !!

```

6. Noms

Choisissez toujours des noms significatifs et non ambigus quant au concept qu'ils désignent. Tenez-vous en à la règles de composition la plus utilisée en Java: majuscules à chaque nouveau mot de la composition (`captainHaddockAge`).

Type	Règles de nommage	Exemples
Classes	La première lettre doit être une majuscule. Évitez les acronymes, choisir des noms simples et descriptifs.	<pre> class Image; class PrivilegedClient; </pre>
Interfaces	Mêmes règles que pour les classes	<pre> interface SingleImage; interface Deposit; </pre>
Méthodes	Les noms de méthodes doivent refléter une action. Choisir de préférence des verbes. Première lettre en minuscules. Première lettre des mots suivants en majuscule.	<pre> display(); getValue(); setValue(); </pre>
Variables	Première lettre en minuscules. Privilégiez la clarté en utilisant des noms longs si besoin. Évitez les noms à une lettre, sauf pour compteurs internes et variables temporaires.	<pre> int i; float width; String nameOfCaptain; </pre>
Constantes	En majuscules. Le séparateur est un souligné.	<pre> final static int MAX_VALUE = 999; </pre>

7. References

[1] Conventions d'écriture de code SUN

<http://java.sun.com/docs/codeconv/html/CodeConventionsTOC.doc.html>

[2] Outil javadoc de SUN

<http://java.sun.com/j2se/javadoc/>